

DNS in Kubernetes

Since you're learning **KCNA as a beginner**, let's understand Kubernetes DNS in a very simple way.

1. What is DNS?

DNS = Domain Name System

Normally, computers communicate using **IP addresses**, but remembering IP addresses is difficult.

For example:

10.96.0.15

Instead of remembering this, DNS lets us use a **name**:

backend-service

So:

backend-service

↓

10.96.0.15

Easy memory:

DNS = Converts names into IP addresses.

2. Why Does Kubernetes Need DNS?

Pods are temporary.

A Pod can be:

Created → Deleted → Recreated

Its IP address may change.

Example:

Old Pod

10.1.0.5



Deleted ❌



New Pod

10.1.0.20

If another application uses the IP directly, it can cause problems.

Kubernetes DNS allows applications to use a **stable name**, usually a Service name.

Frontend Pod



backend-service



Backend Service



Backend Pods

3. Kubernetes DNS Mainly Works with Services ★

Suppose you have:

Backend Service

with:

ClusterIP: 10.96.0.20

Another Pod can access it using:

backend-service

instead of:

10.96.0.20

So:

backend-service



DNS



10.96.0.20



Backend Service

4. CoreDNS ★★ ★

In modern Kubernetes clusters, **CoreDNS** is commonly used to provide cluster DNS.

Think of CoreDNS as the **DNS server of the Kubernetes cluster**.

Pod



| "Where is backend-service?"



CoreDNS



Service IP

🧠 Easy memory:

CoreDNS = Kubernetes DNS server

5. Example

Suppose we have:

Frontend Pod

Backend Service

Backend Pods

Frontend wants to communicate with the backend.

Instead of:

`curl 10.96.0.20`

it can use:

`curl backend-service`

DNS helps resolve:

`backend-service`



Service IP

6. Service DNS Name

Kubernetes creates DNS names for Services.

A common full DNS format is:

`<service-name>.<namespace>.svc.cluster.local`

Example:

`backend-service.default.svc.cluster.local`

Break it down:

`backend-service`



Service name

`default`



Namespace

svc



Service

cluster.local



Cluster DNS domain

 **Remember:**

Service



Namespace



svc



cluster.local

7. Short Name vs Full Name

Suppose:

Service = backend

Namespace = default

Inside the same namespace, you can usually use:

backend

You can also use:

backend.default

And the full DNS name:

backend.default.svc.cluster.local

Easy idea:

Short

backend



Long

backend.default.svc.cluster.local

8. DNS and Namespaces ★

Namespaces help organize Kubernetes resources.

Example:

Namespace: development

└─ backend Service

Namespace: production

└─ backend Service

Both Services can have the same name:

backend

DNS uses the namespace to distinguish them.

backend.development.svc.cluster.local

backend.production.svc.cluster.local

9. Pod → Service → DNS

The most important flow to remember:

Frontend Pod

|

| asks for



backend-service

|



CoreDNS

|



Service IP

|



Backend Pods

10. Does DNS Give the Pod Its IP?

No. ❌

This is an important distinction.

Pod networking

Provides network connectivity and Pod IPs.

CNI



Pod IP

DNS

Helps resolve names.

CoreDNS



Name → IP

So:

 **CNI = Pod networking**

 **DNS = Name resolution**

11. DNS vs Service ★

Don't confuse these.

Service

Provides a stable network endpoint for a group of Pods.

Service



Backend Pods

DNS

Provides a name that can be used to find that Service.

backend-service



DNS



Service

Together:

Application



backend-service



CoreDNS



Service



Backend Pods

12. Kubernetes DNS and External Websites

Kubernetes DNS can also be involved in resolving external domain names, depending on the cluster's DNS configuration.

For example:

Pod



DNS



example.com



External IP

So applications inside Pods can generally resolve normal DNS names as well.

13. Useful Commands

You can check the DNS Pods:

```
kubectrl get pods -n kube-system
```

Look for CoreDNS Pods.

You can also check:

```
kubectrl get deployment -n kube-system
```

You may see:

coredns

14. Simple Testing

You can run a temporary Pod and test DNS:

```
kubectl run dns-test --image=busybox:1.36 --restart=Never --rm -it -- sh
```

Inside the Pod, you can try:

```
nslookup kubernetes.default
```

This tests DNS resolution.

You can also try:

```
nslookup <service-name>
```

if you already have a Service.



KCNA Revision Notes

Kubernetes DNS

- **DNS = Domain Name System.**
- DNS converts/resolves **names to IP addresses.**
- Kubernetes uses DNS for **service discovery.**
- **CoreDNS** commonly provides DNS services in Kubernetes.
- Services get DNS names.
- Applications can use Service names instead of remembering Service IPs.
- A common full Service DNS name is:

service.namespace.svc.cluster.local

- DNS is especially useful because **Pod IPs can change.**
- DNS and CNI are different:
 - **CNI → networking**

- DNS → name resolution

★ Super Easy Memory Trick

🌐 CNI



Gives Pod networking/IP

🔍 CoreDNS



Finds names/IPs

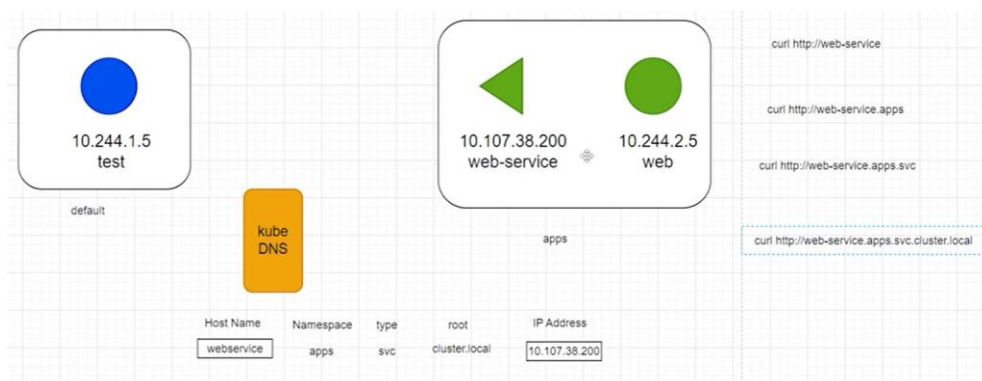
📖 Service



Stable way to reach Pods

One-line definition:

Kubernetes DNS provides name resolution and service discovery so Pods can communicate with Services using names instead of hard-coded IP addresses



Absolutely. This diagram is showing **how Kubernetes DNS allows a Pod to find a Service by name**. Let's go through it step by step.

1. First understand the objects

On the **left**:

Pod: test

IP: 10.244.1.5

Namespace: default

This is a Pod called **test**.

On the **right**:

Service: web-service

Service IP: 10.107.38.200

Namespace: apps

and behind that Service is a Pod:

Pod: web

Pod IP: 10.244.2.5

So the basic structure is:

test Pod

10.244.1.5

|

| wants to contact



web-service

10.107.38.200

|



web Pod

10.244.2.5

2. What is the yellow "kube DNS"?

The yellow box says:

kube

DNS

This represents **Kubernetes DNS**, commonly provided by **CoreDNS**.

Its job is basically:

 **Take a Service name and help resolve it to the Service's IP address.**

For example:

web-service.apps.svc.cluster.local



10.107.38.200

3. Why do we need DNS?

Imagine the test Pod wants to access the web application.

It could use the Service IP:

curl http://10.107.38.200

But remembering IP addresses is inconvenient.

Instead, Kubernetes DNS allows:

curl http://web-service.apps.svc.cluster.local

Much easier! 😊

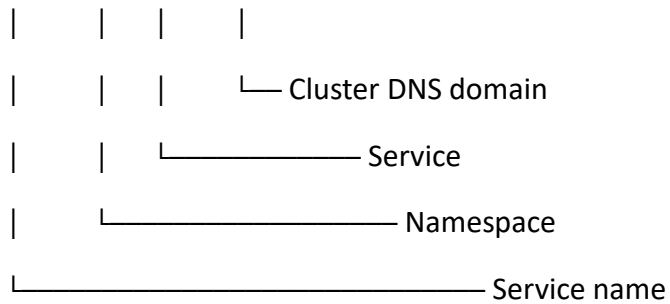
4. Understanding the DNS name ★ ★ ★

The diagram shows:

web-service.apps.svc.cluster.local

This can be broken into 4 parts:

web-service . apps . svc . cluster.local



So:

web-service

→ Name of the Service

apps

→ Namespace where the Service exists

svc

→ Indicates that this is a Service

cluster.local

→ Kubernetes cluster's DNS domain

5. The diagram shows 4 ways to access the Service

On the right side you can see:

`curl http://web-service`

then:

`curl http://web-service.apps`

then:

`curl http://web-service.apps.svc`

and finally:

`curl http://web-service.apps.svc.cluster.local`

These represent increasingly complete DNS names.

6. web-service

```
curl http://web-service
```

This is the **shortest name**.

It works when the client is using the appropriate DNS search configuration, typically when the client is in the same namespace as the Service.

But here there is an important detail:

The test Pod is in:

default

while the Service is in:

apps

So the short name web-service is not the form you'd normally use from the default namespace to reach the Service in apps.

7. web-service.apps

```
curl http://web-service.apps
```

Now we added:

apps

which tells Kubernetes:

"Look for the web-service Service in the apps namespace."

This is useful when accessing a Service in another namespace.

8. web-service.apps.svc

```
curl http://web-service.apps.svc
```

Now we also specify:

svc

which indicates that web-service is a Kubernetes **Service**.

9. Full DNS name ★

curl http://web-service.apps.svc.cluster.local

This is the **fully qualified domain name (FQDN)** shown in your diagram.

Break it down:

web-service



Service name

apps



Namespace

svc



Service

cluster.local



Cluster DNS domain

So the complete meaning is:

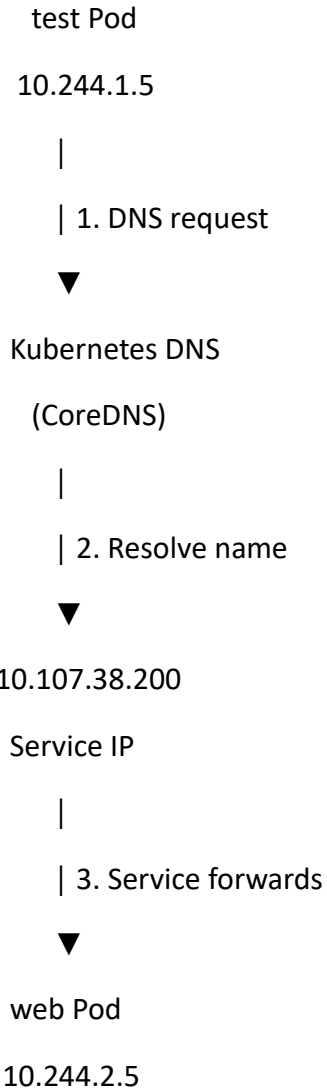
"Find the Service named web-service in the apps namespace inside this Kubernetes cluster."

10. What happens when you run curl?

Suppose the test Pod runs:

curl http://web-service.apps.svc.cluster.local

The basic flow is:



In simple words:

1. test Pod wants to contact web-service.
2. DNS finds the Service's IP.
3. The request goes to the Service.
4. The Service sends the traffic to the appropriate backend Pod.

11. Very important: Service IP vs Pod IP

Your diagram has **two different IP addresses** on the right:

Service:

web-service

10.107.38.200

Pod:

web

10.244.2.5

Don't confuse them! ★

Service IP

10.107.38.200

|



Backend Pod

10.244.2.5

The Service gives you a **stable endpoint**, while Pods can be replaced and their IPs can change.

12. Why is the test Pod in default?

The diagram shows:

test

10.244.1.5

default

So the test Pod belongs to the **default namespace**.

The Service belongs to:

web-service

apps

So it is in the **apps namespace**.

That's why the DNS name includes:

apps

to identify the correct namespace.

Most Important Thing to Remember

Remember this pattern:

`SERVICE-NAME.NAMESPACE.svc.cluster.local`

For your diagram:

`web-service.apps.svc.cluster.local`

means:

web-service



Service name

apps



Namespace

svc



Service

cluster.local



Kubernetes cluster DNS domain

KCNA exam memory:

Kubernetes DNS allows Pods to discover Services using DNS names instead of having to remember Service IP addresses.

And the easiest picture to remember is:

test Pod

|

| web-service.apps.svc.cluster.local



CoreDNS 🔍

|

| resolves name



10.107.38.200

|



web-service

|



web Pod

10.244.2.5

Short memory:

🔍 DNS finds the Service → Service finds the Pod.